

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: Finca Jirah

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Sistema de Gestión Agrícola con los roles Administrador y Agricultor
Operaciones	Gestionar cuenta, Monitorear producción, Ingresar producción campo y Gestionar agricultores
Referencia base	01_Guia_PlantUML_CU_Analisis_CRUD_Jirah
Propósito	Guiar al estudiante para pasar del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso aplicada al Sistema de Gestión Agrícola (SGA). Su finalidad es mostrar cómo los cuatro casos de uso principales pueden analizarse progresivamente mediante tres modelos conectados. El énfasis está en comprender la responsabilidad de cada elemento bajo el paradigma Modelo-Vista-Controlador (MVC), garantizando la trazabilidad desde el requisito hasta la interacción entre objetos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación de caso de uso	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: Sistema de Gestión Agrícola

El sistema gestiona la interacción agrícola. A continuación, se definen los datos principales manejados por el sistema según los perfiles de usuario.

Dato	Tipo sugerido	Descripción	Validación mínima
Agricultor	Texto corto	Cuentas de usuarios gestionadas por el Administrador.	Nombre, cédula, correo y credenciales. Obligatorio y no repetida la cédula.
Registro de la producción recolectada en campo.	Double y entero	Registro de la producción recolectada en campo.	Peso de la gaveta, lote perteneciente Obligatorio. Motivo de rechazo

Rechazos	Texto	Registro de mermas o productos rechazados.	Obligatoria; Motivo del rechazo, lote y comentario.
Preferencias	Configuración de la interfaz del usuario.	Configuración de la interfaz del usuario.	Idioma y tema (claro /oscuro).

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Gestionar Agricultores	Gestionar Agricultores	Crea, edita, elimina y consulta cuentas (CRUD de agricultores).
Ver producción	Monitorear Producción	Visualiza métricas en un dashboard filtrando por fechas, agricultor o rechazos.
Ingresar producción	Ingresar producción campo	Registra el peso de producción, rechazos y sincroniza datos offline/online.
Gestionar cuentas	Gestionar cuenta	Modifica contraseña (validando complejidad) y preferencias de interfaz.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir crear, editar, eliminar y consultar agricultores.	Administrador	Gestionar Agricultores
RF-02	El sistema debe permitir visualizar métricas y filtrar producción.	Administrador	Monitorear Producción
RF-03	El sistema debe permitir registrar producción en campo y rechazos.	Agricultor	Ingresar producción campo
RF-04	El sistema debe permitir modificar contraseñas y preferencias.	Admin / Agricultor	Gestionar cuenta
RF-01	El sistema debe permitir crear, editar, eliminar y consultar agricultores.	Administrador	Gestionar Agricultores

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para el actor. En este caso, el Administrador y agricultor interactúa con las operaciones y la validación se modela como comportamiento obligatorio .

Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador" as Admin
actor "Agricultor" as Agric
```

```

rectangle "Sistema de Gestión Agrícola (SGA)" {
    usecase "Gestionar Agricultores" as UC1
    usecase "Monitorear Producción" as UC2
    usecase "Gestionar Cuenta" as UC3
    usecase "Ingresar producción campo" as UC4
    usecase "Validar Credenciales/Datos" as UC5
}

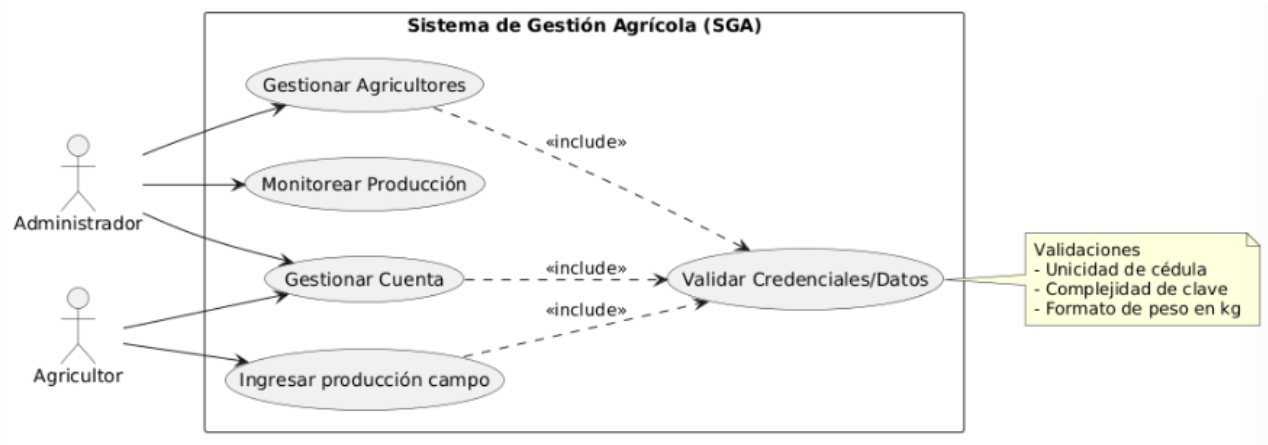
Admin --> UC1
Admin --> UC2
Admin --> UC3

Agric --> UC4
Agric --> UC3

UC1 ..> UC5 : <<include>>
UC3 ..> UC5 : <<include>>
UC4 ..> UC5 : <<include>>

note right of UC5
    Validaciones
    - Unicidad de cédula
    - Complejidad de clave
    - Formato de peso en kg
end note
@enduml

```



6. Derivación de clases de análisis

A partir de los casos de uso se identifican clases de análisis. Para mantener una separación clara de responsabilidades se recomienda usar los estereotipos boundary, control y entity. Además, se puede incluir un repositorio para representar la persistencia o colección de agricultores sin entrar aún en detalles técnicos de base de datos.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaGestionAgricultor, PantallaDashboard, PantallaIngresoCampo, PantallaConfiguracion	Representa las pantallas con campos de ingreso (cédula, peso, lote), filtros y botones de acción (Crear, Guardar, Filtrar).
Control	Coordina la lógica de cada caso de uso.	ControlAgricultor, ControlProduccion, ControlCuenta	Valida datos, decide si crea una cuenta, registra la producción, aplica filtros al dashboard o actualiza la clave.
Entity / Entidad	Representa la información del dominio.	Agricultor, Produccion, Cuenta	Contiene los atributos principales (nombre, cédula, peso, lote, preferencias).
Repositorio / Colección	Conserva y recupera registros.	RepositorioSGA	Permite guardar, buscar, actualizar, sincronizar y listar datos agrícolas y usuarios.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

package "VISTA (Boundary)" {
    class "VistaGestionAgricultores" <<boundary>> {
        + capturarDatosCreacion()
        + capturarDatosEdicion()
        + capturarCriterioBusqueda()
        + seleccionarEliminacion()
        + mostrarMensaje(msg: String)
    }
    class "VistaMonitoreoProduccion" <<boundary>> {
        + seleccionarFiltros(rango, motivo, agric)
        + renderizarDashboard(datos: List)
    }
    class "VistaIngresoProduccion" <<boundary>> {
        + ingresarPeso()
        + ingresarLote()
        + reportarMotivoRechazo()
    }
    class "VistaGestionCuenta" <<boundary>> {
        + ingresarNuevaContrasena()
        + seleccionarPreferencias()
    }
}

package "CONTROLADOR (Control)" {
    class "ControladorAgricultores" <<control>> {
```

```

+ crearAgricultor(datos: Map)
+ consultarAgricultor(criterio: String): Agricultor
+ editarAgricultor(datos: Map)
+ eliminarAgricultor(cedula: String)
- validarCedulaUnica(cedula: String): boolean
}

class "ControladorMonitoreo" <<control>> {
    + aplicarFiltros(filtros: Map): List
}

class "ControladorProduccion" <<control>> {
    + registrarProduccion(peso: double, lote: String)
    + registrarRechazo(motivo: String, lote: String)
    + sincronizarOffline()
}

class "ControladorCuenta" <<control>> {
    + modificarContrasena(actual: String, nueva: String)
    + modificarPreferencias(idioma: String, tema: String)
    - validarComplejidad(clave: String): boolean
}
}

package "MODELO (Entity/Repository)" {
    class "Agricultor" <<entity>> {
        - cedula: String
        - nombre: String
        - correo: String
    }

    class "Produccion" <<entity>> {
        - pesoGaveta: double
        - lote: String
        - motivoRechazo: String
        - fecha: Date
    }

    class "Cuenta" <<entity>> {
        - contrasenaHash: String
        - idiomaPreferido: String
        - temaInterfaz: String
    }

    class "RepositorioSGA" <<repository>> {
        + guardarAgricultor(a: Agricultor)
        + actualizarAgricultor(a: Agricultor)
        + borrarAgricultor(cedula: String)
        + obtenerAgricultor(criterio: String): Agricultor
        + guardarProduccion(p: Produccion)
        + consultarProduccionFiltros(f: Map): List
        + actualizarCuenta(c: Cuenta)
    }
}
}

```

```

VistaGestionAgricultores --> ControladorAgricultores
VistaMonitoreoProduccion --> ControladorMonitoreo
VistaIngresoProduccion --> ControladorProduccion
VistaGestionCuenta --> ControladorCuenta

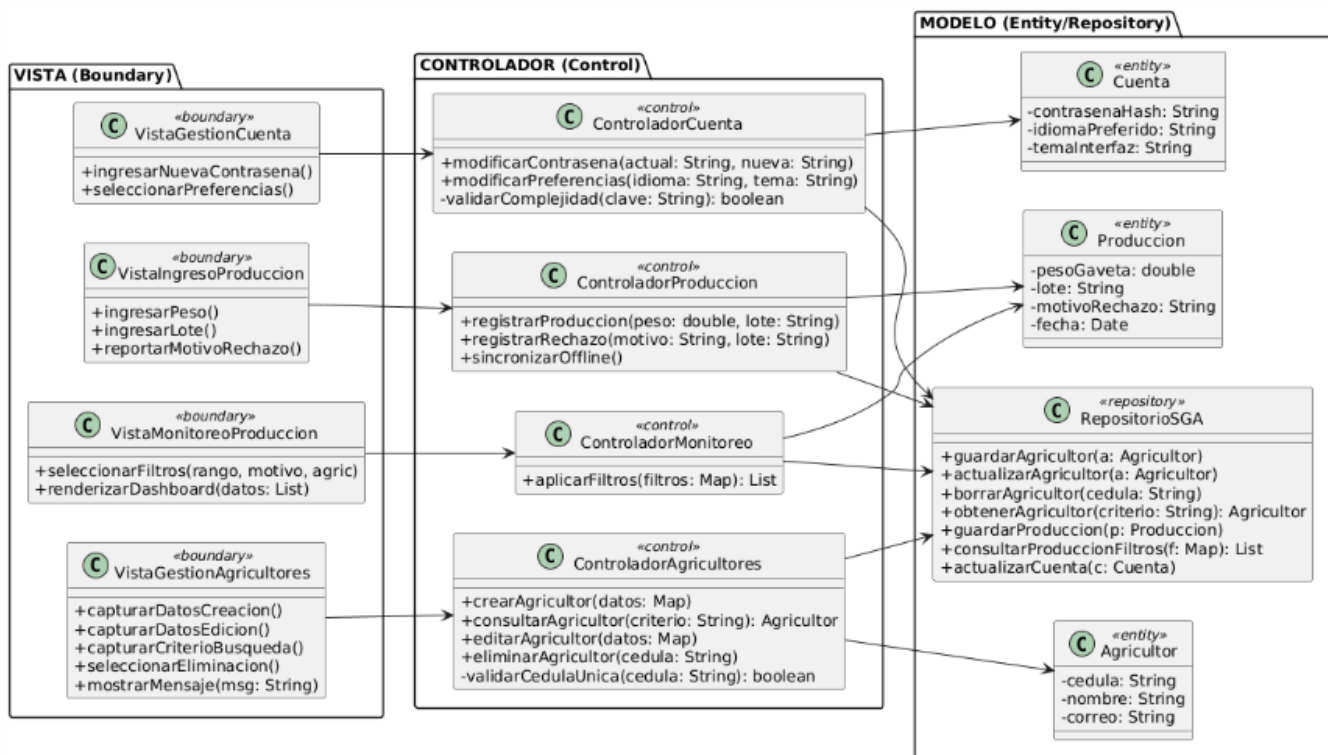
ControladorAgricultores --> Agricultor
ControladorAgricultores --> RepositorioSGA

ControladorMonitoreo --> Produccion
ControladorMonitoreo --> RepositorioSGA

ControladorProduccion --> Produccion
ControladorProduccion --> RepositorioSGA

ControladorCuenta --> Cuenta
ControladorCuenta --> RepositorioSGA
@enduml

```



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Gestionar Agricultores	VistaGestionAgricultores captura datos de creación, edición o búsqueda.	ControladorAgricultores ejecuta métodos del CRUD (crear, editar, eliminar, consultar).	Entidad Agricultor. RepositorioSGA persiste los	4 Secuencias Crear, Consultar,

			cambios del CRUD.	Editar y Eliminar agricultor.
Monitorear Producción	VistaMonitoreoProduccion captura filtros (fechas/motivos).	ControladorMonitoreo procesa los filtros solicitados.	Entidad Produccion. RepositorioSGA ejecuta la consulta.	Aplicar filtros y visualizar métricas.
Ingresar Producción Campo	VistaIngresoProduccion captura peso, lote o motivos de rechazo.	ControladorProduccion coordina validación de peso y sincronización.	Entidad Produccion. RepositorioSGA guarda el registro.	Registrar peso/rechazo en campo.
Gestionar Cuenta	VistaGestionCuenta captura nueva clave o temas de interfaz.	ControladorCuenta verifica complejidad y políticas.	Entidad Cuenta. RepositorioSGA actualiza los datos.	Modificar Contraseña y Modificar Preferencias.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se recomienda elaborar un diagrama por cada caso de uso principal para evitar diagramas sobrecargados.

8.1 Secuencia: Gestionar Agricultores

A) Crear Agricultor

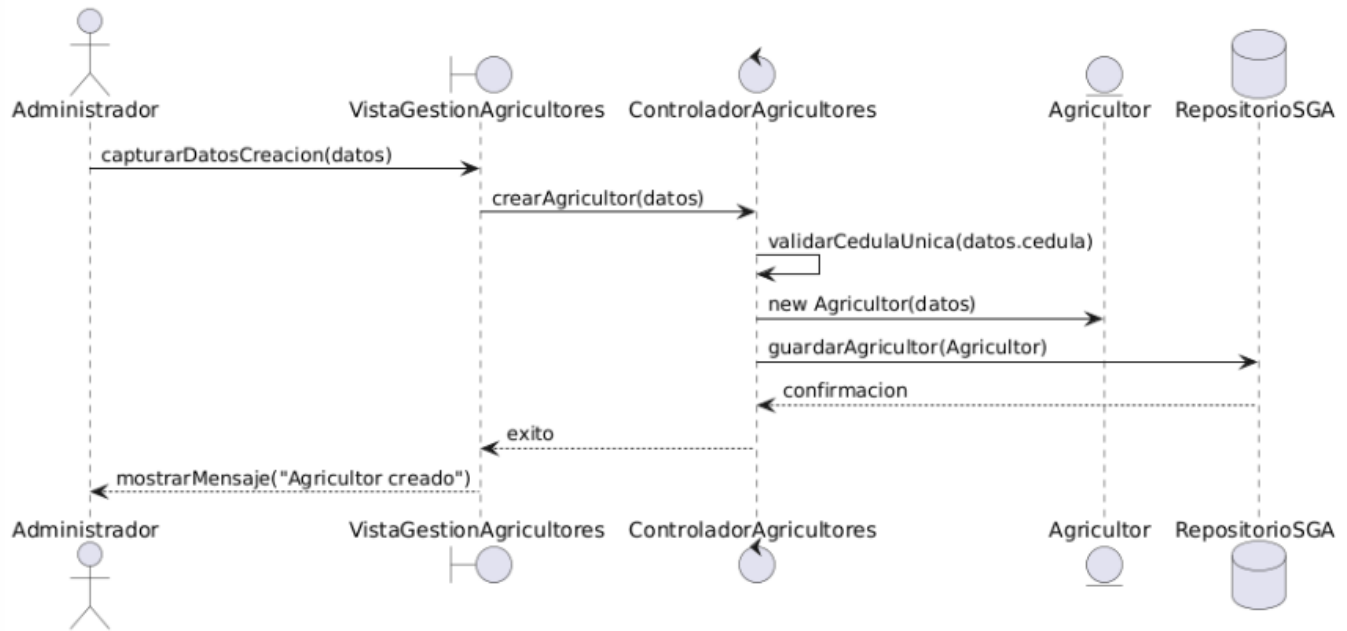
```

@startuml
title Secuencia - Crear Agricultor
actor Administrador as Admin
boundary VistaGestionAgricultores as Vista
control ControladorAgricultores as Ctrl
entity Agricultor as ModeloEnt
database RepositorioSGA as Repo

Admin -> Vista : capturarDatosCreacion(datos)
Vista -> Ctrl : crearAgricultor(datos)
Ctrl -> Ctrl : validarCedulaUnica(datos.cedula)
Ctrl -> ModeloEnt : new Agricultor(datos)
Ctrl -> Repo : guardarAgricultor(Agricultor)
Repo --> Ctrl : confirmacion
Ctrl --> Vista : exito
Vista --> Admin : mostrarMensaje("Agricultor creado")
@enduml

```

Secuencia - Crear Agricultor



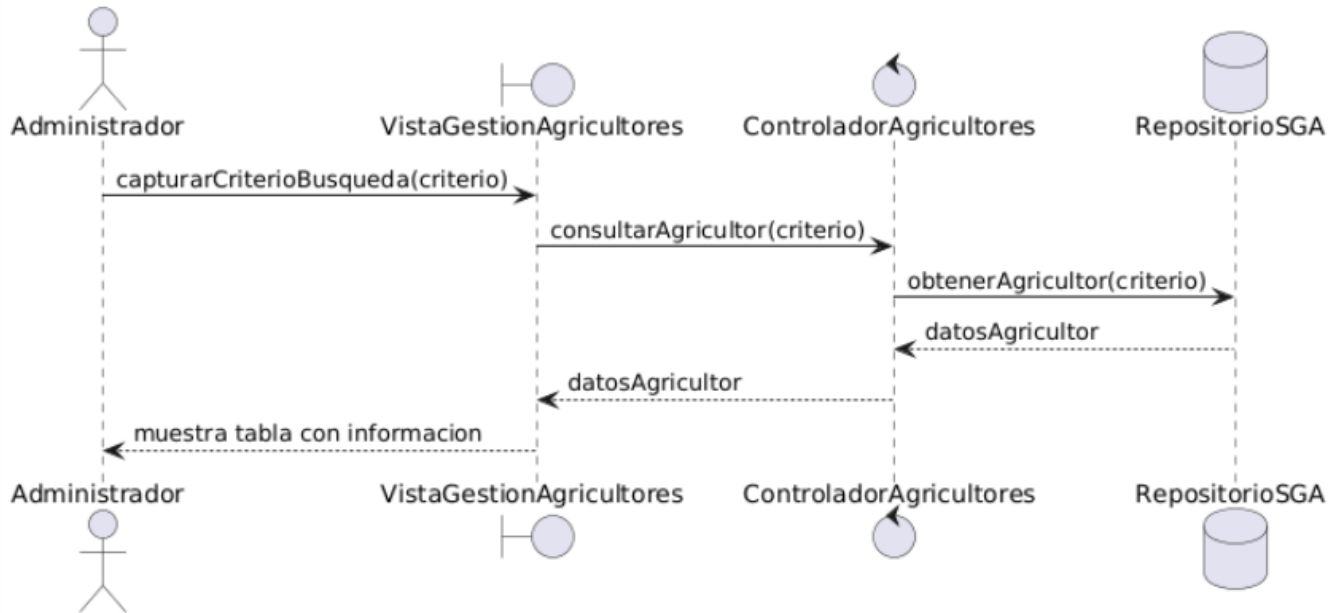
B) Consultar Agricultor

```

@startuml
title Secuencia - Consultar Agricultor
actor Administrador as Admin
boundary VistaGestionAgricultores as Vista
control ControladorAgricultores as Ctrl
database RepositorioSGA as Repo

Admin -> Vista : capturarCriterioBusqueda(criterio)
Vista -> Ctrl : consultarAgricultor(criterio)
Ctrl -> Repo : obtenerAgricultor(criterio)
Repo --> Ctrl : datosAgricultor
Ctrl --> Vista : datosAgricultor
Vista --> Admin : muestra tabla con informacion
@enduml
    
```

Secuencia - Consultar Agricultor



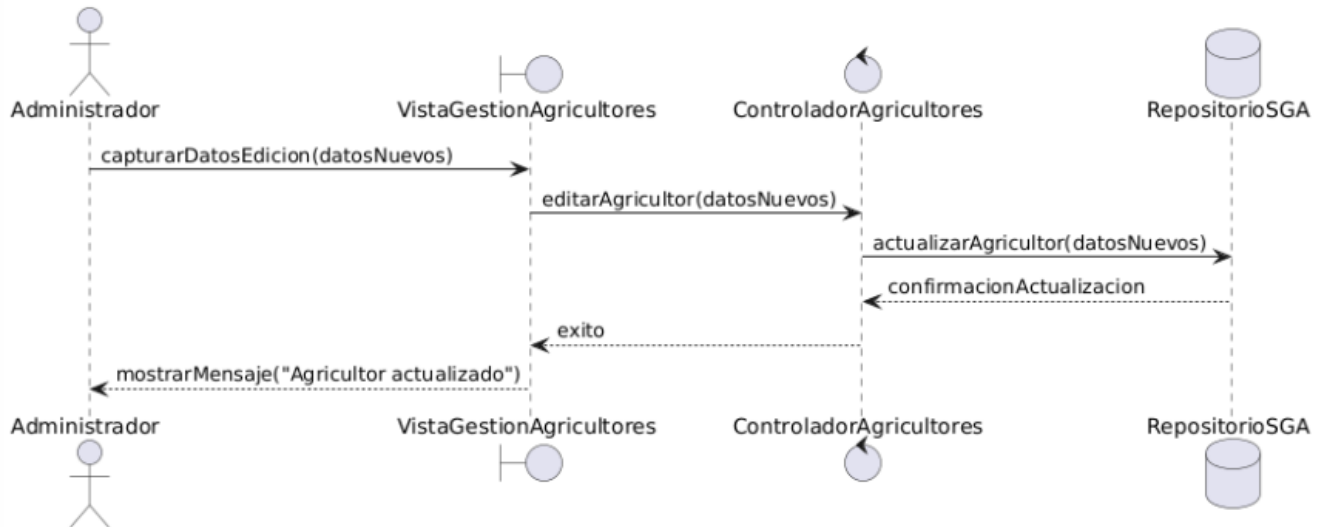
C) Editar Agricultor

```

@startuml
title Secuencia - Editar Agricultor
actor Administrador as Admin
boundary VistaGestionAgricultores as Vista
control ControladorAgricultores as Ctrl
database RepositorioSGA as Repo

Admin -> Vista : capturarDatosEdicion(datosNuevos)
Vista -> Ctrl : editarAgricultor(datosNuevos)
Ctrl -> Repo : actualizarAgricultor(datosNuevos)
Repo --> Ctrl : confirmacionActualizacion
Ctrl --> Vista : exito
Vista --> Admin : mostrarMensaje("Agricultor actualizado")
@enduml
    
```

Secuencia - Editar Agricultor



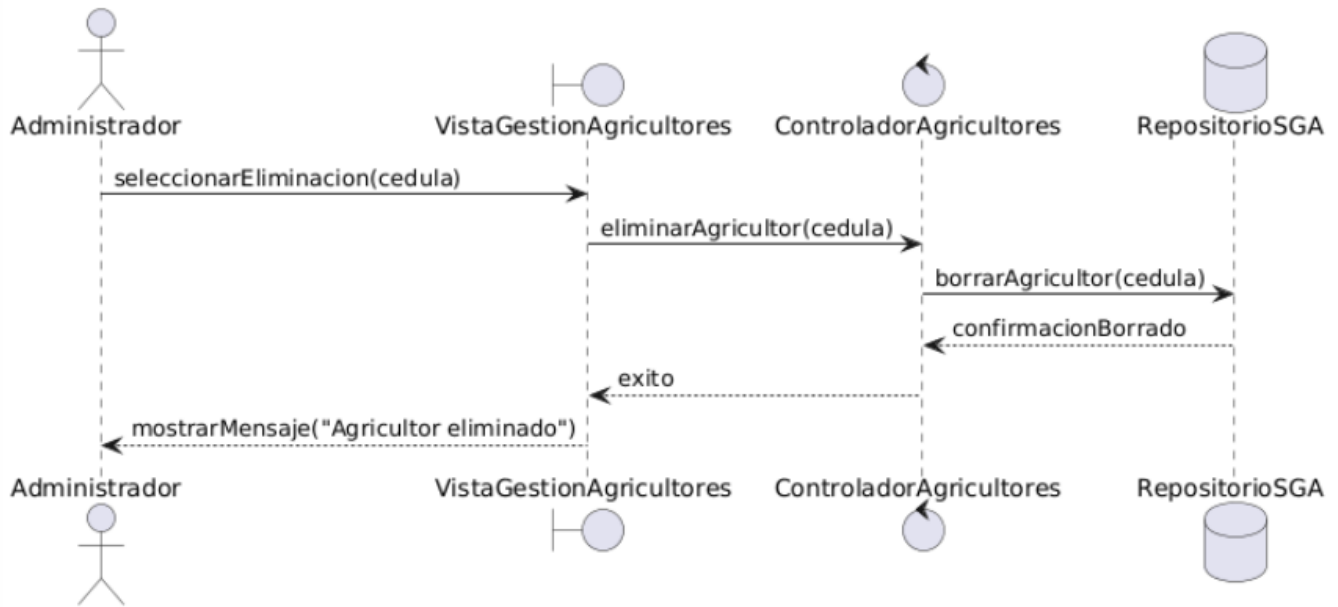
D) Eliminar Agricultor

```

@startuml
title Secuencia - Eliminar Agricultor
actor Administrador as Admin
boundary VistaGestionAgricultores as Vista
control ControladorAgricultores as Ctrl
database RepositorioSGA as Repo

Admin -> Vista : seleccionarEliminacion(cedula)
Vista -> Ctrl : eliminarAgricultor(cedula)
Ctrl -> Repo : borrarAgricultor(cedula)
Repo --> Ctrl : confirmacionBorrado
Ctrl --> Vista : exito
Vista --> Admin : mostrarMensaje("Agricultor eliminado")
@enduml
    
```

Secuencia - Eliminar Agricultor



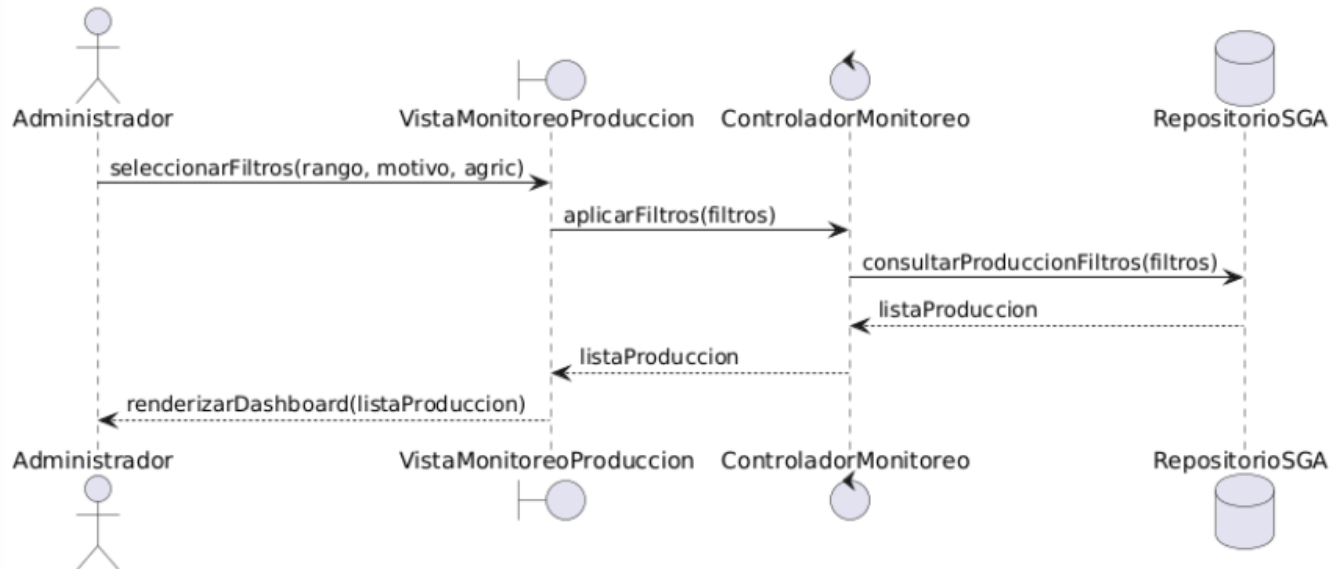
8.2 Secuencia: Monitorear Producción

```

@startuml
title Secuencia - Monitorear Producción
actor Administrador as Admin
boundary VistaMonitoreoProduccion as Vista
control ControladorMonitoreo as Ctrl
database RepositorioSGA as Repo

Admin -> Vista : seleccionarFiltros(rango, motivo, agric)
Vista -> Ctrl : aplicarFiltros(filtros)
Ctrl -> Repo : consultarProduccionFiltros(filtros)
Repo --> Ctrl : listaProduccion
Ctrl --> Vista : listaProduccion
Vista --> Admin : renderizarDashboard(listaProduccion)
@enduml
    
```

Secuencia - Monitorear Producción



8.3 Secuencia: Ingresar producción campo

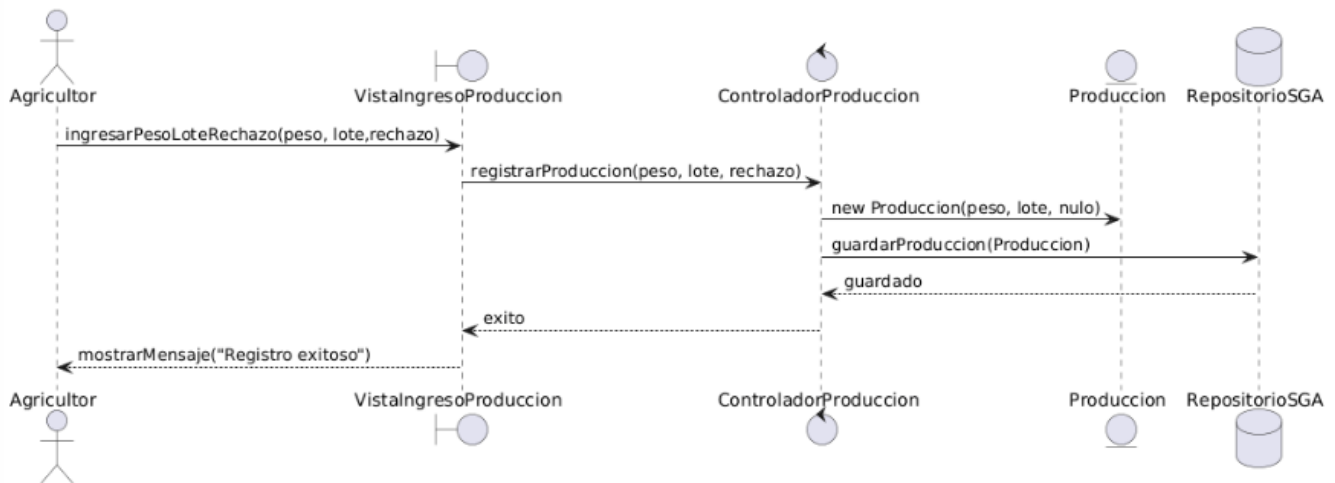
```

@startuml
title Secuencia - Ingresar Producción Campo
actor Agricultor as Agric
boundary VistaIngresoProduccion as Vista
control ControladorProduccion as Ctrl
entity Produccion as ModeloEnt
database RepositorioSGA as Repo

Agric -> Vista : ingresarPesoLoteRechazo(peso, lote, rechazo)
Vista -> Ctrl : registrarProduccion(peso, lote, rechazo)
Ctrl -> ModeloEnt : new Produccion(peso, lote, nulo)
Ctrl -> Repo : guardarProduccion(Produccion)
Repo --> Ctrl : guardado
Ctrl --> Vista : exito
Vista --> Agric : mostrarMensaje("Registro exitoso")
@enduml

```

Secuencia - Ingresar Producción Campo



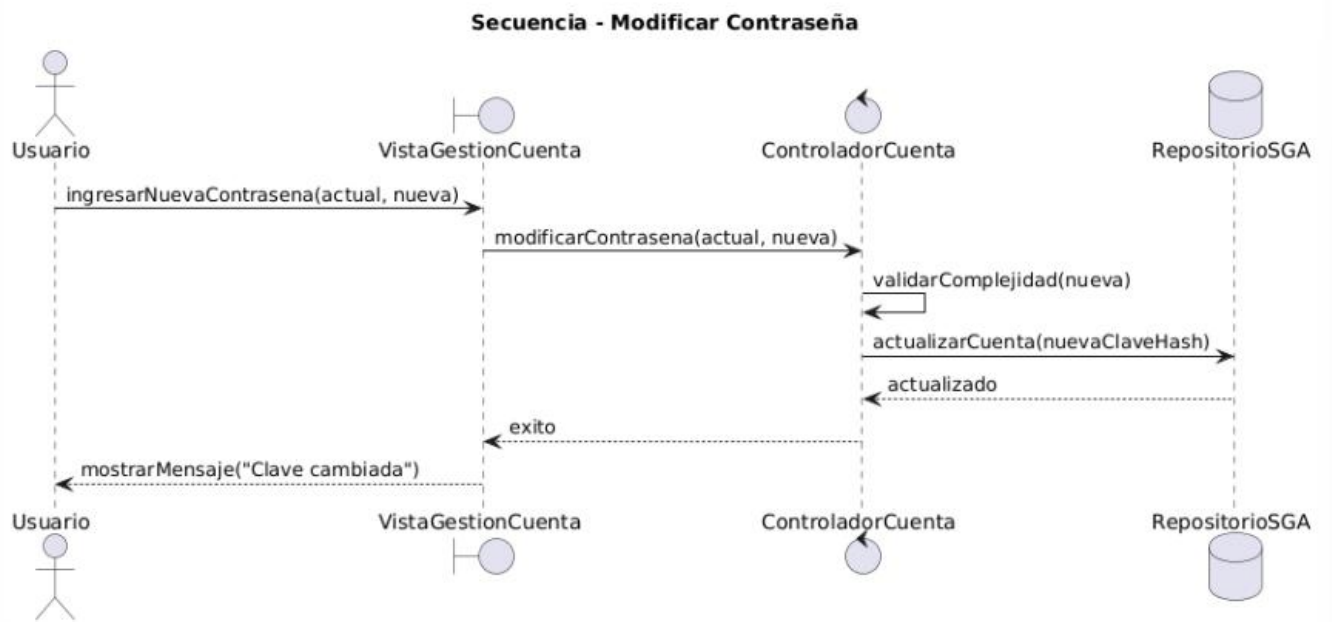
8.4 Secuencia: Gestionar Cuenta

A) Modificar Contraseña

```

@startuml
title Secuencia - Modificar Contraseña
actor Usuario as User
boundary VistaGestionCuenta as Vista
control ControladorCuenta as Ctrl
database RepositorioSGA as Repo

User -> Vista : ingresarNuevaContrasena(actual, nueva)
Vista -> Ctrl : modificarContrasena(actual, nueva)
Ctrl -> Ctrl : validarComplejidad(nueva)
Ctrl -> Repo : actualizarCuenta(nuevaClaveHash)
Repo --> Ctrl : actualizado
Ctrl --> Vista : exito
Vista --> User : mostrarMensaje("Clave cambiada")
@enduml
    
```

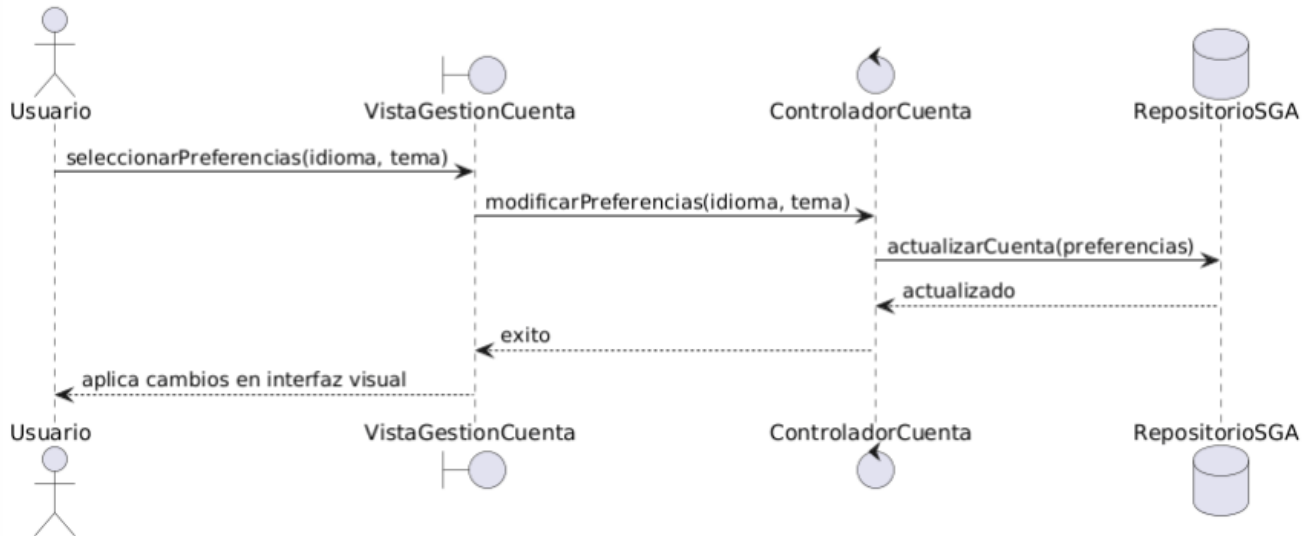


B) Modificar Preferencias

```

@startuml
title Secuencia - Modificar Contraseña
actor Usuario as User
boundary VistaGestionCuenta as Vista
control ControladorCuenta as Ctrl
database RepositorioSGA as Repo

User -> Vista : ingresarNuevaContrasena(actual, nueva)
Vista -> Ctrl : modificarContrasena(actual, nueva)
Ctrl -> Ctrl : validarComplejidad(nueva)
Ctrl -> Repo : actualizarCuenta(nuevaClaveHash)
Repo --> Ctrl : actualizado
Ctrl --> Vista : exito
Vista --> User : mostrarMensaje("Clave cambiada")
@enduml
    
```

Secuencia - Modificar Preferencias**9. Especificación breve de casos de uso conectada con las secuencias**

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Gestionar agricultores	El administrador ha iniciado sesión y la cédula no debe existir previamente (para creación).	Ingresar datos (nombre, cédula, correo) -> validar datos -> verificar cédula única -> guardar/actualizar agricultor.	Cuenta de agricultor creada/actualizada correctamente y visible en el sistema.
Monitorear producción	El administrador ha iniciado sesión y existen datos de producción registrados.	Seleccionar dashboard -> aplicar filtros (fechas, agricultor, rechazos) -> recuperar registros -> mostrar gráficos.	Dashboard actualizado en tiempo real con las métricas correspondientes.
Ingresar producción campo	El agricultor dispone de los datos de recolección en campo.	Ingresar peso y lote (o motivo de rechazo) -> validar datos -> guardar registro (o sincronizar si estaba offline).	Producción o rechazo registrado exitosamente en el sistema.
Gestionar cuenta	El usuario (Administrador o Agricultor) ha iniciado sesión.	Ingresar nueva contraseña o seleccionar preferencias -> validar políticas de seguridad -> guardar cambios.	Contraseña o preferencias actualizadas correctamente para futuras sesiones.

10. Procedimiento recomendado para el estudiante

- Revisar los perfiles de usuario (SGA) y los campos (Peso, Lote, Cédula, Correo).
- Redactar requisitos funcionales de nivel 0 para los 4 flujos de trabajo.
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones include necesarias.
- Derivar clases bajo el patrón MVC: identificar clases <<boundary>> (Vistas), <<control>> (Controladores lógicos) y <<entity>>/<<repository>> (Modelos de datos).
- Construir un diagrama de secuencia por cada caso de uso principal.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.

- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de los controladores y repositorios.
- La validación de datos se aplica antes de agregar y actualizar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones incluye justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para Agregar, Actualizar, Eliminar y Mostrar Todo.	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La relación entre casos de uso, diagrama de clases y diagramas de secuencia permite validar la arquitectura de software de forma temprana. En el Sistema de Gestión Agrícola, tener en cuenta siempre la trazabilidad en la etapa de análisis ha garantizado que las responsabilidades no se acoplen: las vistas se limitan a recibir interacciones del Agricultor o Administrador, los controladores concentran la validación de negocio y los modelos se encargan únicamente de estructurar y persistir los datos de la finca.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.